Customer Churn Analysis

Do the customer attributes we hold identify who will leave?

Case study | SQL (MySQL), Python and Tableau | Dataset: ecommerce_customer_data.csv, 2,000 customers | October 2026

> **Bottom line.** Across seven one-at-a-time tests, 31 customer segments and a logistic regression model, **no customer attribute showed a statistically detectable link to churn**. The model's cross-validated AUC was **0.496** (0.5 means chance), indistinguishable from models trained on shuffled labels (empirical p = 0.645), and its top-10% lift was **0.93**. The recommendation is not to build a churn-targeting model on these features, but to run a randomized retention-offer pilot and to collect richer behavioral data.

In plain language: we tested whether the customer details we have could predict who will leave, and they could not do better than guessing.

1. Problem Statement

An online retailer lost 31.05% of the customers in this dataset (621 of 2,000). The question is whether the customer information already on file (age, gender, income, spending score, membership length, online purchases and discount usage) can identify which customers are likely to churn, and whether churn differs across customer segments.

Why it matters

Retention offers such as discounts, outreach and loyalty perks cost money. If customer attributes cannot tell the team who is at risk, targeting by those attributes spends budget without improving outcomes. A model that looks accurate can also mislead: because 69% of customers do not churn, a model that predicts “no churn” for everyone is right 69% of the time without learning anything about who leaves.

Who is affected

- **The retention and marketing team**, who choose whom to contact and with what offer.
- **Finance stakeholders**, who approve retention spend and need to know whether it can be targeted to pay back.
- **Customers**, who receive (or do not receive) offers based on how they are classified.

> **The decision this analysis informs:** Should the retention team use these customer attributes to choose who receives a retention offer?

2. Goals and Success Measures

The project aims to answer the decision above honestly, with evidence that separates real patterns from random variation. Specifically, it sets out to:

- audit the data and reconcile every count between SQL and Python;
- test whether churn differs by customer attribute, with uncertainty reported for every comparison;
- build a churn model and test it against what chance alone produces;
- communicate the result in a dashboard that cannot be misread as a ranking of “worst” segments;

- turn the result into a recommendation that stays within what the evidence supports.

Success is judged against thresholds that were **fixed before the results were seen**, so that the analysis could not be shaped to fit the outcome:

Measure	Threshold or rule	Why this value
Data reconciliation	Segment counts sum to 2,000; count-weighted gaps sum to zero; SQL and pandas counts agree	A lost or double-counted customer breaks the arithmetic
Feature tests (7)	$p < 0.05 / 7 \approx 0.0071$ (Bonferroni)	Seven tests inflate false positives to about 30%
Segment intervals (31)	99.84% intervals ($0.05 / 31 \approx 0.0016$)	31 segments on one screen
Model discrimination	Mean cross-validated AUC > 0.55	About 2.5 SD above what shuffled labels produce
Targeting value	Lift at top 10% > 1.24	1.0 plus two standard errors for a 160-customer slice
Final test set	Scored once; expected AUC 0.44 to 0.56	0.5 ± 2 SD for 400 customers (SD ≈ 0.031)

3. Context

Customer churn means a customer stops buying or ends their membership. Retailers usually weigh retention spending against the alternative of replacing lost customers, so the practical question is whether an offer costs less than the value of the customer it keeps. That is only answerable if the team can tell which customers are at risk and whether an offer changes their behavior.

Three statistical ideas shape how this analysis is built:

- **Base rate.** About 31% of customers churn, so any prediction has to beat that background rate, not just look accurate.
- **Sampling noise.** With 2,000 customers, a segment of 200 has a churn rate that can move by several points from chance alone. A gap that looks large in a table can be within noise.
- **Multiple comparisons.** The more slices of the data are examined, the more likely one looks interesting by luck. With 15 buckets the chance that at least one looks “significant” under no effect is $1 - 0.95^{15} \approx 53.7\%$; for the nine age bands it is about 37%, and for the 31 segments on the dashboard about 80%.

4. Data Description

The file contains **2,000 customers and 9 columns**, one row per customer.

Variable	Type	Description and observed range
Customer_ID	Identifier	Unique per customer; excluded from all models
Age	Integer	Documented as 18 to 70; observed 18 to 69
Gender	Categorical	Female (1,044) or Male (956)
Annual_Income	Numeric	Yearly income in USD
Spending_Score	Numeric	Score from 1 to 100 based on spending behavior
Membership_Years	Numeric	Years as a customer; 0.5 to 10.0 in steps of 0.1
Online_Purchases	Integer	Total number of online purchases
Discount_Usage	Numeric	Share of discount usage, 0 to 1
Churn	Binary target	0 = no churn (1,379 customers), 1 = churn (621 customers)

Quality checks

- **No missing values** in any column, and **no duplicate customer IDs**.
- `Churn` contains only 0 and 1 (validated in SQL and pandas).
- **Moderate class imbalance:** 31.05% churn, so predicting "no churn" for everyone scores 68.95% accuracy.
- **Features are nearly uncorrelated with each other** (largest pairwise correlation among the six numeric features: −0.042), so multicollinearity is not a concern for the model.
- **Provenance is undocumented.** The file does not say how it was collected. Initial profiling found near-zero correlations between every feature and churn (absolute r ≤ 0.033), which is consistent with randomly generated data. This limits what any conclusion can claim about a real customer base.

5. Data Preparation

Preparation had two parts: building a clean, validated table in MySQL for descriptive work, and reproducing the same segmentation in Python so the two could be checked against each other.

5.1 Loading and validating the table (SQL)

The CSV was imported with MySQL Workbench's import wizard. The row count had to equal 2,000 before any analysis continued.

```
CREATE DATABASE churn_case;
USE churn_case;

CREATE TABLE customers (
    Customer_ID      INT PRIMARY KEY,
    Age              TINYINT UNSIGNED,
    Gender           VARCHAR(10),
    Annual_Income    INT,
    Spending_Score   TINYINT UNSIGNED,
    Membership_Years DECIMAL(3,1),
    Online_Purchases INT,
    Discount_Usage   DECIMAL(3,2),
    Churn            TINYINT
);

SELECT COUNT(*) FROM customers;   -- expect 2000 before continuing
```

The validation query separates NULLs from invalid values on purpose. In SQL, `NULL NOT IN (0, 1)` evaluates to NULL, not TRUE, and `SUM()` ignores NULLs, so a missing churn value would never appear in an invalid-value count without its own NULL check.

```
SELECT
    COUNT(*)                  AS total_rows,
    SUM(Churn IS NULL)        AS null_count,
    SUM(Churn NOT IN (0, 1))  AS invalid_values
FROM customers;
-- result: 2000, 0, 0
```

5.2 Overall churn rate

`AVG(Churn)` gives the churn rate directly because the mean of a 0/1 column is the proportion of 1s. That is safe only when each row is one observation, the coding is 0/1, there are no NULLs, and rates are not averaged across groups of different sizes. If 200 rows had NULL churn, `AVG()` would divide by the 1,800 non-NULL rows while `COUNT(*)` would count all 2,000, and the two methods would silently disagree (for 621 churned customers, 34.5% against 31.05%). The validation query above shows that this does not occur here.

```
SELECT
    COUNT(*) AS total_customers,
    SUM(CASE WHEN Churn = 1 THEN 1 ELSE 0 END) AS total_churn_customers,
    ROUND(AVG(Churn) * 100, 1) AS churn_rate_pct
FROM customers;
-- result: 2000, 621, 31.1  (31.05% before rounding)
```

5.3 Churn by membership tenure

Membership_Years is a decimal, so integer-style buckets such as 0–2, 3–5, 6–8 leave gaps (a customer with 2.5 years fits none of them). The first draft used `BETWEEN` and an `ELSE '10+'` label, which silently relabeled those customers as “10+”. The fix was to use `>=` on the lower bound and `<` on the upper bound, so adjacent buckets share a boundary and every value lands in exactly one bucket, and to relabel the fallback `'UNBUCKETED'` so that any gap shows up as a visible row. Labels also had to describe their contents: a bucket holding 2.5–5.4 years cannot be called “3–5”.

```
WITH membership_bucket AS (
    SELECT CASE WHEN membership_years >= 0   AND membership_years < 2.5 THEN '0-2.5'
                WHEN membership_years >= 2.5 AND membership_years < 5.5 THEN '2.5-5.5'
                WHEN membership_years >= 5.5 AND membership_years < 8.5 THEN '5.5-8.5'
                WHEN membership_years >= 8.5 AND membership_years <= 10 THEN '8.5-10'
                ELSE 'UNBUCKETED' END AS membership_band,   -- diagnostic: must stay empty
           Churn
    FROM customers)
SELECT membership_band,
       COUNT(*) AS total_customers,
       ROUND(AVG(Churn) * 100, 1) AS membership_churn
FROM membership_bucket
GROUP BY membership_band
ORDER BY membership_band;
```

Tenure bucket (years)	Customers	Churned	Churn rate	Gap vs overall (pts)
0 to < 2.5	420	144	34.3%	+3.2
2.5 to < 5.5	641	189	29.5%	−1.6
5.5 to < 8.5	603	187	31.0%	0.0
8.5 to 10	336	101	30.1%	−1.0

Reconciliation: the four counts sum to 2,000, the churned counts sum to 621, and the count-weighted average of the bucket rates reproduces the overall 31.05% (to within rounding), with no `UNBUCKETED` row.

5.4 Comparing each bucket with the overall rate (window functions)

To add an all-customer rate to every grouped row, the query carries counts, not just rates, in a second CTE and applies `SUM(...) OVER ()` over the grouped rows.

```
WITH membership_bucket AS (
    SELECT
        CASE WHEN membership_years >= 0   AND membership_years < 2.5 THEN '0-2.5'
             WHEN membership_years >= 2.5 AND membership_years < 5.5 THEN '2.5-5.5'
             WHEN membership_years >= 5.5 AND membership_years < 8.5 THEN '5.5-8.5'
             WHEN membership_years >= 8.5 AND membership_years <= 10 THEN '8.5-10'
             ELSE 'UNBUCKETED' END AS membership_band,
        CASE WHEN membership_years >= 0   AND membership_years < 2.5 THEN 1
             WHEN membership_years >= 2.5 AND membership_years < 5.5 THEN 2
             WHEN membership_years >= 5.5 AND membership_years < 8.5 THEN 3
             WHEN membership_years >= 8.5 AND membership_years <= 10 THEN 4
             ELSE 5 END AS sort_order,                        -- numeric sort key
        Churn
    FROM customers
),
bucket_stats AS (                                             -- one row per bucket
    SELECT membership_band, sort_order,
           COUNT(*)   AS total_customers,
           SUM(Churn) AS churn_count
    FROM membership_bucket
    GROUP BY membership_band, sort_order
)
SELECT
    membership_band, total_customers, churn_count,
    ROUND(100 * churn_count / total_customers, 1) AS bucket_churn_pct,
    ROUND(100 * SUM(churn_count) OVER ()
              / SUM(total_customers) OVER (), 1) AS overall_churn_pct,
    ROUND(100 * (churn_count / total_customers
                 - SUM(churn_count) OVER () / SUM(total_customers) OVER ()), 1) AS gap_pp
FROM bucket_stats
ORDER BY sort_order;
```

> **A bug worth recording.** An earlier version divided each bucket's churn total by `COUNT(*) OVER()`. That function counted **4 rows**, because `GROUP BY` had already collapsed 2,000 customers into four bucket rows, so each "rate" was a churn count divided by 4 (144 → 36.00, 189 → 47.25, 187 → 46.75, 101 → 25.25). `SUM(churn_count) OVER()` instead added up the four churn counts to give 621. The rule: **a window function operates on the rows that exist at that stage of the query, not on the rows in the table.**

A related check confirms the design. The count-weighted sum of the gaps must be zero, because the overall rate is itself the customer-weighted average of the bucket rates: $\Sigma\, n_i \times (\text{rate}_i - \text{overall}) = 621 - 0.3105 \times 2{,}000 = 0$. The unweighted sum is not zero (about +0.64 points), which is the weighted-versus-unweighted trap in action.

5.5 Age bands in SQL and Python

Age was split into nine six-year bands (18–23 through 66–70), using the same two-CTE pattern.

```
WITH age_bucket AS (
    SELECT
        CASE WHEN Age BETWEEN 18 AND 23 THEN '18-23'
             WHEN Age BETWEEN 24 AND 29 THEN '24-29'
             WHEN Age BETWEEN 30 AND 35 THEN '30-35'
             WHEN Age BETWEEN 36 AND 41 THEN '36-41'
             WHEN Age BETWEEN 42 AND 47 THEN '42-47'
             WHEN Age BETWEEN 48 AND 53 THEN '48-53'
             WHEN Age BETWEEN 54 AND 59 THEN '54-59'
             WHEN Age BETWEEN 60 AND 65 THEN '60-65'
             WHEN Age BETWEEN 66 AND 70 THEN '66-70'
             ELSE 'UNBUCKETED' END AS age_band,
        CASE WHEN Age BETWEEN 18 AND 23 THEN 1  WHEN Age BETWEEN 24 AND 29 THEN 2
             WHEN Age BETWEEN 30 AND 35 THEN 3  WHEN Age BETWEEN 36 AND 41 THEN 4
             WHEN Age BETWEEN 42 AND 47 THEN 5  WHEN Age BETWEEN 48 AND 53 THEN 6
```

```
            WHEN Age BETWEEN 54 AND 59 THEN 7  WHEN Age BETWEEN 60 AND 65 THEN 8
            WHEN Age BETWEEN 66 AND 70 THEN 9  ELSE 10 END AS sort_order,
        Churn
    FROM customers
),
bucket_stats AS (
    SELECT age_band, sort_order, COUNT(*) AS total_customers, SUM(Churn) AS churn_count
    FROM age_bucket GROUP BY age_band, sort_order
)
SELECT age_band, total_customers, churn_count,
       ROUND(100 * churn_count / total_customers, 1) AS bucket_churn_pct,
       ROUND(100 * SUM(churn_count) OVER ()
                 / SUM(total_customers) OVER (), 1) AS overall_churn_pct,
       ROUND(100 * (churn_count / total_customers
                    - SUM(churn_count) OVER () / SUM(total_customers) OVER ()), 1) AS gap_pp
FROM bucket_stats
ORDER BY sort_order;
```

The same bands were rebuilt in pandas with `pd.cut`. Each edge must be the first age of a band, `right=False` makes intervals left-closed, and the last edge is 70 so that age 69 is included. A first attempt used edges 52, 58, 64 for three bands and shifted every band after 48, which the NaN check cannot catch (all ages still fall in some band); only the count comparison with SQL exposes it.

```
import numpy as np
import pandas as pd
from scipy import stats

df = pd.read_csv("ecommerce_customer_data.csv")          # relative path: reproducible

print(df.shape)                                          # (2000, 9)
print(df.isnull().sum())                                 # 0 in every column
print("Duplicate IDs:", df["Customer_ID"].duplicated().sum())   # 0
print(df["Churn"].value_counts())                        # 0: 1379, 1: 621

# Each edge is the FIRST age of a band; right=False makes the bands [18,24), [24,30), ...
edges  = [18, 24, 30, 36, 42, 48, 54, 60, 66, 70]
labels = ["18-23","24-29","30-35","36-41","42-47","48-53","54-59","60-65","66-69"]
df["age_band"] = pd.cut(df["Age"], bins=edges, labels=labels, right=False)

assert df["age_band"].isna().sum() == 0  # Python version of 'UNBUCKETED' check

table = pd.crosstab(df["age_band"], df["Churn"])         # reproduces the SQL counts exactly
chi2, p, dof, expected = stats.chi2_contingency(table)  # order: statistic, p-value, dof, expected
print(round(chi2, 3), round(p, 3), dof, expected.min().round(1))   # 4.62  0.797  8  51.2

# Tie-back: the statistic is the sum of squared z-scores of each band against the overall rate
p0 = df["Churn"].mean()
g  = df.groupby("age_band", observed=True)["Churn"].agg(n="count", churned="sum")
g["z"] = (g["churned"] / g["n"] - p0) / np.sqrt(p0 * (1 - p0) / g["n"])
print((g["z"] ** 2).sum().round(4))                      # 4.6197
```

The pandas crosstab reproduced the SQL customer counts (232, 217, 216, 217, 260, 248, 225, 220, 165) and churn counts (67, 60, 78, 67, 80, 76, 69, 72, 52) exactly.

6. Method

6.1 Descriptive segmentation, with uncertainty

Churn rates were compared across segments of every feature. Age and tenure use fixed bands; income, spending score, online purchases and discount usage use quartiles (about 500 customers each); gender has two groups. That is **31 segments**. Segment definitions were set from the features alone, before any churn rate was examined, so that bands could not be tuned to produce a pattern.

Band-width trade-off. Wider bands give more customers per group, which shrinks the standard error and reduces the number of comparisons, but may hide a real pattern inside a band. Narrower bands give

more detail but noisier estimates and more chances for a large-looking gap to occur by chance. The nine age bands (165 to 260 customers each) have standard errors of about 3 percentage points, against about 2.3 for the larger tenure buckets.

6.2 One-at-a-time hypothesis tests

Feature type	Primary test	Why
Numeric (six features)	Welch's t-test, churned minus not churned	Compares means without assuming equal variances; features were not skewed (|skew| < 0.08)
Gender (2 groups)	Chi-square on a 2×2 table (Yates correction)	Two categorical variables; the uncorrected version is a sensitivity check
Multiplicity	Bonferroni: 0.05 / 7 ≈ 0.0071	Seven features tested

Rule for in-between results. A p-value between 0.0071 and 0.05 is reported as "nominally significant at 0.05 but not after correction for seven tests". It is not treated as a strong finding, and no recommendation is built on it without further validation on new data. With seven true-null tests, the chance that at least one p-value lands below 0.05 is about 30% ($1 - 0.95^7$), so a result in that range is expected fairly often by chance.

Interpreting a p-value. The p-value is the probability of a result at least this extreme if the null hypothesis were true. A high p-value means the observed differences could reasonably have occurred by chance if there were no relationship. It is not the probability that there is no relationship, and it does not prove that none exists.

```
features = ["Age", "Annual_Income", "Spending_Score",
            "Membership_Years", "Online_Purchases", "Discount_Usage"]
ALPHA = 0.05 / 7  # Bonferroni threshold, fixed in advance
rows = []

for col in features:  # six numeric features: Welch t-test
    churned = df.loc[df["Churn"] == 1, col]              # churned FIRST, every time
    kept    = df.loc[df["Churn"] == 0, col]
    t  = stats.ttest_ind(churned, kept, equal_var=False)
    ci = t.confidence_interval()                         # interval for (churned - kept)
    diff = churned.mean() - kept.mean()
    assert np.sign(t.statistic) == np.sign(diff)         # guards against a flipped sign
    sd = df[col].std()
    rows.append({"feature": col, "test": "Welch t-test", "effect": diff,
                 "ci_low": ci.low, "ci_high": ci.high,
                 "ci_low_sd": ci.low / sd, "ci_high_sd": ci.high / sd,
                 "statistic": t.statistic, "p_value": t.pvalue})

tab = pd.crosstab(df["Gender"], df["Churn"])             # Gender: chi-square on a 2x2 table
chi2, p, dof, _ = stats.chi2_contingency(tab, correction=True)   # Yates (SciPy default)
rate = tab[1] / tab.sum(axis=1)
d    = rate["Male"] - rate["Female"]
se   = np.sqrt((rate * (1 - rate) / tab.sum(axis=1)).sum())
rows.append({"feature": "Gender", "test": "Chi-square (Yates)", "effect": d,
             "ci_low": d - 1.96 * se, "ci_high": d + 1.96 * se,
             "ci_low_sd": np.nan, "ci_high_sd": np.nan, "statistic": chi2, "p_value": p})

results = pd.DataFrame(rows)
results["below_threshold"] = results["p_value"] < ALPHA
print(results.round(4))
```

6.3 Predictive model

A logistic regression predicts churn from all seven features together, which the one-at-a-time tests cannot assess. Preprocessing follows from how the model works:

- **Gender** is one-hot encoded with `drop="first"`, giving one binary column (`Gender_Male`, with Female as the reference). Keeping both columns alongside an intercept would create perfect redundancy (the dummy-variable trap).
- **Numeric features are standardized.** A coefficient is the change in log-odds for a one-unit increase, and one unit of Annual_Income is one dollar while one unit of Discount_Usage spans its entire 0 to 1 range. Standardizing makes each coefficient “per one standard deviation”. It also affects the fit itself, because scikit-learn’s default L2 penalty treats unscaled features unevenly.
- **Scaling would not matter for a tree-based model**, because trees split on the ordering of values, which standardizing does not change.

Metrics. Accuracy cannot show that the model learned anything, since “predict no churn” already scores 69%. **AUC** is the probability that a randomly chosen churned customer receives a higher predicted churn probability than a randomly chosen non-churned customer (ties count as half). An AUC of 0.5 means chance, and a model that gives every customer the same score gets exactly 0.5 because it cannot rank anyone. As a second metric, **Precision@K** answers the business question “of the K customers ranked most likely to churn, what fraction really churn?”, and **lift** divides that by the 31% baseline (1.0 means no better than random targeting).

Evaluation design. The data was split 80/20 (stratified). Repeated stratified cross-validation (5 folds × 10 repeats) ran on the training part only, because a single 80/20 split would leave only about 125 churners in the test set and make the AUC sensitive to which customers landed there. Scoring a model on the customers it trained on gives an optimistic AUC. The test set was used once, at the end.

```
from sklearn.model_selection import (train_test_split, RepeatedStratifiedKFold,
                                     StratifiedKFold, cross_val_score, cross_val_predict)
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

numerical_features   = ["Annual_Income", "Spending_Score", "Membership_Years",
                        "Online_Purchases", "Discount_Usage", "Age"]
categorical_features = ["Gender"]  # Customer_ID is an identifier: excluded
X = df[numerical_features + categorical_features]
y = df["Churn"]

# 80/20 split, stratified so both parts keep the 31% churn rate. Test set used ONCE, at the end.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

# Scaling and encoding sit INSIDE the pipeline, so each CV fold is scaled using its own
# training rows only (no leakage). Scaling also matters because LogisticRegression applies
# an L2 penalty by default, which treats unscaled features unevenly.
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numerical_features),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), categorical_features),
])
pipeline = Pipeline([("preprocessor", preprocessor), ("model", LogisticRegression())])

# Repeated stratified CV on the REAL training labels: 5 folds x 10 repeats = 50 AUC scores
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=42)
cv_scores = cross_val_score(pipeline, X_train, y_train, cv=cv, scoring="roc_auc")
print("Mean CV AUC:", cv_scores.mean(), "| SD:", cv_scores.std())   # 0.4960 | 0.0281

# Guard: stops the run if cv_scores was ever computed on shuffled labels
assert abs(cv_scores.mean() - 0.4960) < 0.001, "cv_scores changed: restart and run all cells"

# Precision@K: out-of-fold predictions (single StratifiedKFold = one prediction per customer)
cv_pred = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
y_prob = cross_val_predict(pipeline, X_train, y_train, cv=cv_pred,
                           method="predict_proba")[:, 1]       # column 1 = P(churn)
```

```
results = pd.DataFrame({"actual_churn": np.asarray(y_train), "churn_probability": y_prob})
k = int(0.10 * len(results))                                  # top 10% = 160 customers
results = results.sort_values("churn_probability", ascending=False)
precision_at_k = results.head(k)["actual_churn"].mean()       # 0.2875
baseline       = results["actual_churn"].mean()               # 0.3106
lift_at_k      = precision_at_k / baseline                    # 0.93

# FINAL EVALUATION: the single use of the test set
pipeline.fit(X_train, y_train)
y_test_prob = pipeline.predict_proba(X_test)[:, 1]
test_auc = roc_auc_score(y_test, y_test_prob)                 # 0.487

# Verdict against the bars set in advance
mean_ok, lift_ok = cv_scores.mean() > 0.55, lift_at_k > 1.24
print("Mean CV AUC", "Passed" if mean_ok else "Failed", "(bar 0.55)")
print(f"Lift@{k}", "Passed" if lift_ok else "Failed", "(bar 1.24)")
```

6.4 Segment intervals for the dashboard

For each of the 31 segments, the dashboard needs (a) the segment's own churn rate with a Wilson interval, and (b) the gap between the segment and **all other customers**, with a Newcombe interval for a difference of two independent proportions. Comparing with everyone else, not the overall rate, removes the segment's own contribution from the baseline. Intervals are Bonferroni-adjusted for 31 comparisons (each at the 99.84% level). The full export script is in Appendix A.

7. Validation

7.1 Reconciliation checks

- SQL and pandas agree on all nine age-band customer counts and churn counts, and on the 2,000 / 621 totals.
- Every feature's segments sum to 2,000 customers (asserted in the export script), and the count-weighted gaps sum to zero.
- A guard (`assert`) stops the model script if the cross-validated mean is ever computed on shuffled labels.

7.2 Worked example: the age chi-square and its z-score tie-back

A chi-square test across the nine age bands gave a statistic of **4.62** with **8 degrees of freedom** and **p = 0.797**. Under no relationship the statistic averages its degrees of freedom (8) with a standard deviation of 4, so 4.62 is ordinary. All expected counts exceed 5 (minimum about 51), so the approximation is reliable. The statistic is the sum of each band's squared z-score against the overall rate, which reproduces it exactly (4.6197):

Age band	Customers	Churned	Churn rate	Gap vs overall	z	Share of Σz²
18–23	232	67	28.9%	−2.2 pts	−0.71	11%
24–29	217	60	27.6%	−3.4 pts	−1.08	25%
30–35	216	78	36.1%	+5.1 pts	+1.61	56%
36–41	217	67	30.9%	−0.2 pts	−0.06	0%
42–47	260	80	30.8%	−0.3 pts	−0.10	0%
48–53	248	76	30.6%	−0.4 pts	−0.14	0%
54–59	225	69	30.7%	−0.4 pts	−0.12	0%
60–65	220	72	32.7%	+1.7 pts	+0.54	6%
66–69	165	52	31.5%	+0.5 pts	+0.13	0%

The 30–35 band is the most extreme ($z \approx 1.6$, below the 2 conventionally used as a threshold), and it was picked because it was the largest of nine gaps. Selecting the most extreme of nine after looking breaks the assumption behind a z-score, which presumes the band was chosen in advance. A rough estimate of how often the most extreme of nine bands looks this extreme under no effect is about 65% (treating the bands as independent).

7.3 Model validation against the bars set in advance

Check	Bar	Result	Outcome
Mean cross-validated AUC	> 0.55	0.496 (SD 0.028 across 50 folds)	Not met
Lift at top 10% (160 customers)	> 1.24	0.93 (precision 28.8% vs baseline 31.1%)	Not met
Test-set AUC (400 customers, scored once)	Expected 0.44 to 0.56	0.487 (z = −0.42)	Inside expected range

The lift of 0.93 is about 0.6 standard errors below 1.0 (a slice of 160 has a standard error of about 0.12 in lift units), so it is within noise. It does not show that the model ranks customers backwards. The test-set result agrees with cross-validation but is one 400-customer sample, so it adds confirmation, not independent weight.

7.4 Permutation check: what does noise score?

To test whether the 0.55 bar was a sensible one, the churn labels in the training set were shuffled 30 times (breaking any real link to the features) and the same cross-validation was run on each. The shuffled means centered on **0.503** with a standard deviation of **0.019** (range 0.472 to 0.546). **None of the 30 reached 0.55**, though the highest came within 0.004. Nineteen of the 30 were at or above the real model's 0.496, giving an empirical p-value of (19 + 1) / (30 + 1) = **0.645**: the real model sits in the middle of the noise.

With the observed spread, 0.55 is about 2.5 standard deviations above the shuffled center, so noise would reach it roughly once in 170 runs. That makes the bar reasonable but less strict than the 3.2 standard deviations the analytic formula suggested. Thirty shuffles cannot distinguish a 0.1% tail from a 1% tail, so more shuffles (about 200) would be needed to pin this down. Even in the best case, 30 shuffles can only produce a p-value as small as 1/31 ≈ 0.03.

```
# PERMUTATION CHECK: what does pure noise score? Uses only X_train, y_train, pipeline, cv.
shuffled_means = []
for seed in range(30):                                  # a different shuffle each pass
    y_shuffled = np.random.default_rng(seed).permutation(y_train)  # break the link to churn
    scores = cross_val_score(pipeline, X_train, y_shuffled, cv=cv, scoring="roc_auc")
    shuffled_means.append(scores.mean())                # one number per shuffle
```

```
shuffled_means = np.array(shuffled_means)

real_mean     = cv_scores.mean()                        # 0.496, from the real-label run
n_at_or_above = (shuffled_means >= real_mean).sum()     # 19 of 30
p_empirical   = (n_at_or_above + 1) / (len(shuffled_means) + 1)   # (19 + 1) / 31 = 0.645
print("Reaching 0.55:", (shuffled_means >= 0.55).sum(), "of", len(shuffled_means))  # 0 of 30
```

7.5 Bootstrap check on the model coefficients

The training data was resampled with replacement 500 times, refitting the whole pipeline each time, to see how stable each coefficient is. All coefficients are per one standard deviation, except `Gender_Male`, which is a 0/1 contrast with Female as the reference. The bootstrap standard deviations match the analytic approximation $1 / \sqrt{(n \times p \times (1 - p))} \approx 0.054$ for standardized features (and about 0.11 for the 0/1 gender column).

```
from sklearn.base import clone

B = 500
rng = np.random.default_rng(42)                         # created once, before the loop
coef_vectors = []
for _ in range(B):
    positions = rng.integers(0, len(X_train), size=len(X_train))   # rows WITH replacement
    X_boot, y_boot = X_train.iloc[positions], y_train.iloc[positions]  # same rows for X and y
    boot_model = clone(pipeline).fit(X_boot, y_boot)     # scaler and model both refit
    coef_vectors.append(boot_model.named_steps["model"].coef_[0])
coef_array = np.vstack(coef_vectors)                    # shape (500, 7)

names = pipeline.named_steps["preprocessor"].get_feature_names_out()  # needs fitted pipeline
coef_summary = pd.DataFrame({
    "Feature": names, "Mean": coef_array.mean(axis=0),
    "SD": coef_array.std(axis=0, ddof=1),
    "CI_2.5%": np.percentile(coef_array, 2.5, axis=0),
    "CI_97.5%": np.percentile(coef_array, 97.5, axis=0)})
coef_summary["Includes_zero"] = ((coef_summary["CI_2.5%"] <= 0)
                                 & (coef_summary["CI_97.5%"] >= 0))
```

Feature	Mean	SD	2.5th pct	97.5th pct	Zero?
Annual_Income	+0.0060	0.0539	−0.1011	+0.1164	Yes
Spending_Score	−0.0754	0.0546	−0.1746	+0.0320	Yes
Membership_Years	−0.0814	0.0530	−0.1840	+0.0157	Yes
Online_Purchases	+0.0158	0.0542	−0.0975	+0.1207	Yes
Discount_Usage	−0.0848	0.0537	−0.1848	+0.0201	Yes
Age	+0.0289	0.0505	−0.0701	+0.1258	Yes
Gender_Male (vs Female)	+0.0331	0.1102	−0.1872	+0.2315	Yes

All seven intervals include zero, the most likely outcome (about 70%) if no feature has an effect. Gender's coefficient of +0.033 corresponds to an odds ratio of about 1.03, with an interval of roughly 0.83 to 1.26. That is a statement about **odds**, not probability, and the interval is too wide to distinguish a decrease from an increase.

7.6 Checks for bias and overfitting

- **Leakage:** scaling and encoding sit inside the cross-validation pipeline; `Customer_ID` is excluded; the test set is touched once.
- **Overfitting:** all performance numbers come from held-out folds or the held-out test set, never from the training data.
- **Selection bias in segments:** definitions were fixed before churn rates were seen, and every segment is reported, not just the interesting ones.

- **Multiple comparisons:** Bonferroni thresholds were set for both families (0.0071 for seven features, 0.0016 for 31 segments) before the results were seen. The two thresholds differ because they correct different families.

8. Visualizations and Findings

8.1 Seven features, seven tests

Each primary test compares churned with non-churned customers (effect = churned minus not churned; Gender = male minus female churn rate). The last column puts each 95% interval on a common scale by dividing by the feature's standard deviation.

Feature	Test	Effect	95% CI	p-value	< 0.0071	CI in SD units
Age	Welch	+0.52 yrs	−0.89 to +1.93	0.473	No	−0.06 to +0.13
Annual_Income	Welch	−$514	−$4,059 to +$3,032	0.776	No	−0.11 to +0.08
Spending_Score	Welch	−1.39	−4.08 to +1.31	0.312	No	−0.14 to +0.05
Membership_Years	Welch	−0.15 yrs	−0.42 to +0.11	0.250	No	−0.15 to +0.04
Online_Purchases	Welch	+0.53	−4.95 to +6.00	0.850	No	−0.09 to +0.10
Discount_Usage	Welch	−0.020	−0.047 to +0.006	0.134	No	−0.17 to +0.02
Gender	Chi-sq (Yates)	+0.2 pts	−3.8 to +4.3 pts	0.949	No	n/a

None of the seven comes close to the 0.0071 threshold; the smallest p-value (Discount_Usage, 0.134) is about 19 times larger. The uncorrected Gender chi-square (0.0126, p = 0.910) leads to the same conclusion. Every standardized bound lies within about ±0.17 standard deviations, so the data rules out moderate or large differences (about 0.2 SD or more, by Cohen's convention) but cannot rule out small ones.

8.2 Segment view: churn rates with uncertainty

Figure 1 shows the churn rate of each of the 31 segments as a zero-based bar with a 95% Wilson whisker, the segment size in the axis label and the overall rate (31.05%) as a reference line. Segments stay in natural order (age young to old, tenure short to long, quartiles Q1 to Q4) and are never sorted by churn rate, so a viewer can see whether a trend exists. All 31 whiskers span the overall rate.

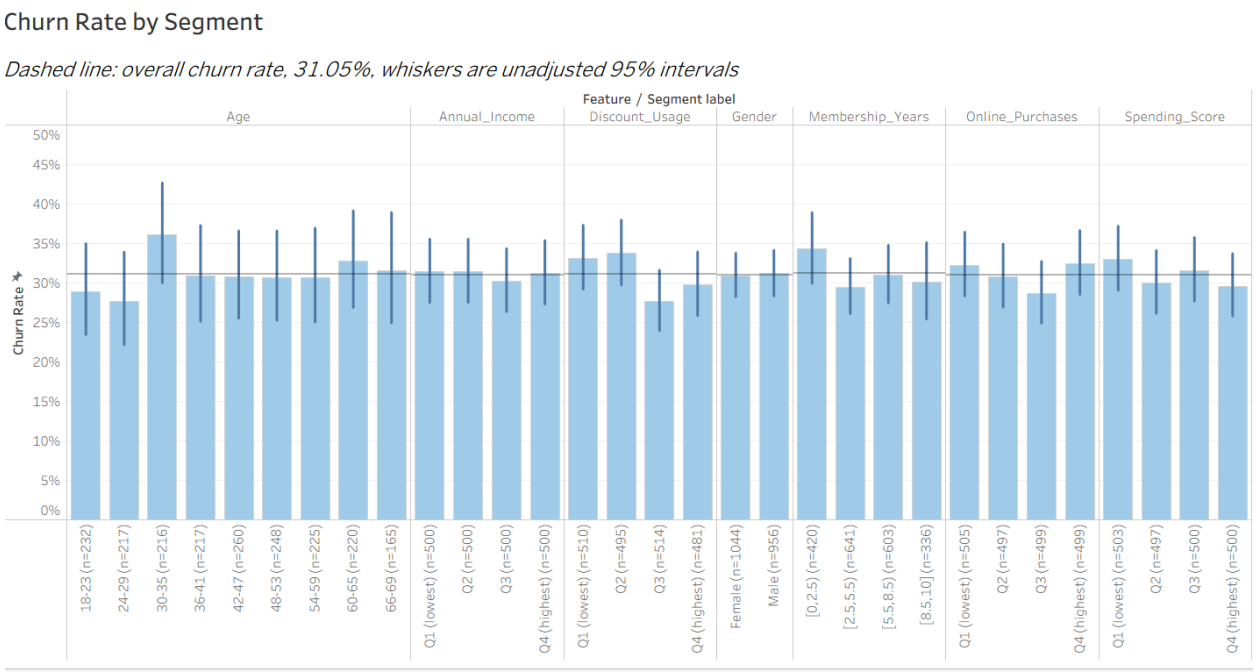


Figure 1. Churn rate by segment (Tableau). Bars start at zero, whiskers are unadjusted 95% intervals, and the line marks the overall rate of 31.05%.

Within each feature the bars do not follow a steady trend. The nearest to one is Spending_Score (33.0%, 30.0%, 31.6%, 29.6%, falling overall but not monotone), and Discount_Usage swings from 33.7% in Q2 to 27.6% in Q3. A real effect would change smoothly across ordered segments, and a jump up followed by a jump down is what noise looks like.

8.3 Segment view: gap against all other customers

Figure 2 plots each segment's gap from everyone else, with Bonferroni-adjusted 99.84% intervals, against a zero line. **Zero of 31 adjusted intervals exclude zero**, and zero of 31 exclude it even at the unadjusted 95% level, where about 1.6 would be expected by chance.

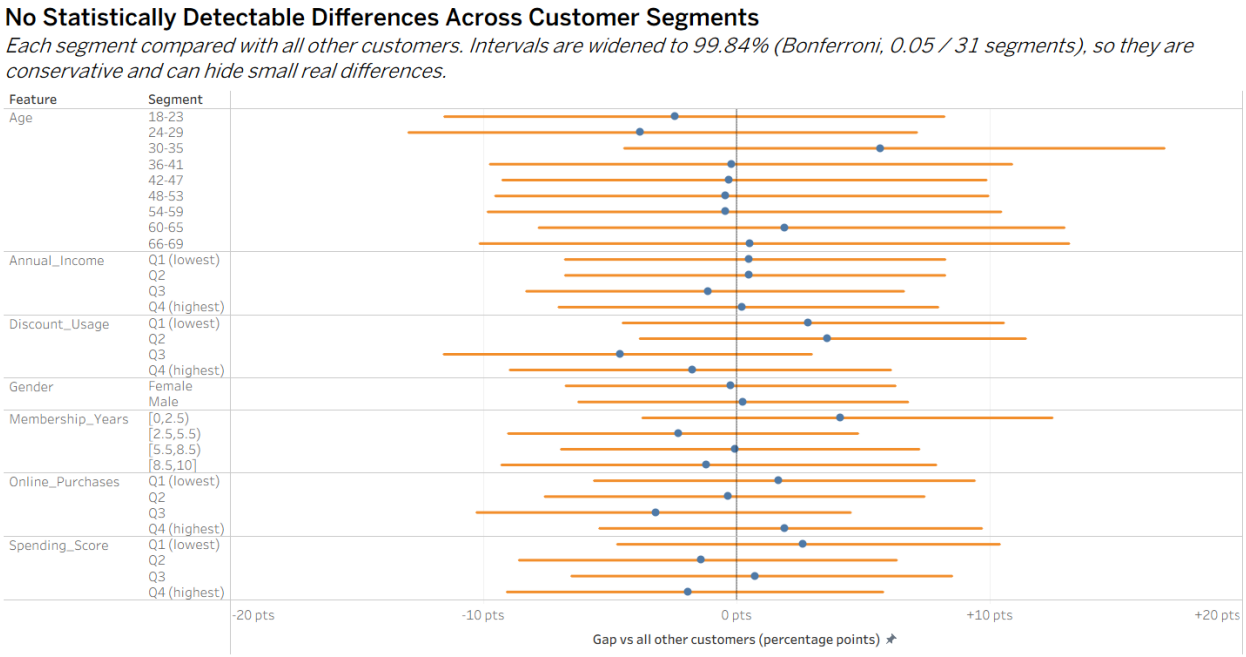


Figure 2. Gap in churn rate between each segment and all other customers, in percentage points (Tableau). Every interval crosses zero.

The four closest calls (unadjusted intervals) are Discount_Usage Q3 (gap −4.6 points, upper bound essentially at zero), Age 30–35 (+5.7, −0.8 to +12.6), tenure under 2.5 years (+4.1, −0.9 to +9.3) and Discount_Usage Q2 (+3.6, −1.1 to +8.4). With 31 comparisons, one or two such near-misses are expected by chance. The two Gender rows are mirror images of a single comparison, so the effective family is 30, and keeping the correction at 31 is slightly conservative.

8.4 Model view: the real AUC against shuffled labels

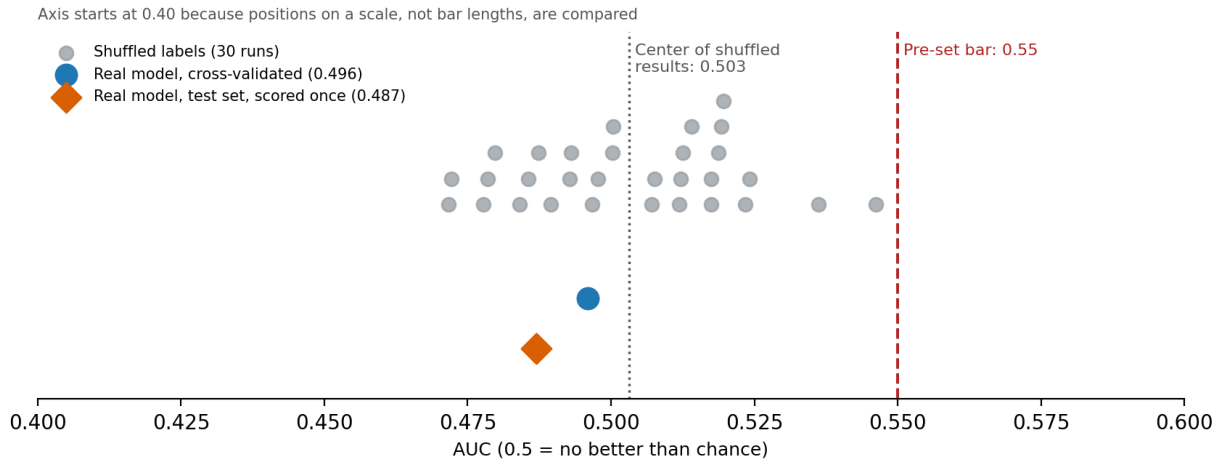


Figure 3. Cross-validated AUC of the real model against 30 label-shuffled runs. The real result sits inside the cloud of noise-only results.

Figure 3 is the strongest single exhibit, because it does not depend on how segments were cut. The real model's cross-validated AUC (0.496) and its single test-set score (0.487) both fall among the shuffled results, and none of the 30 shuffled runs reached the pre-set bar of 0.55.

8.5 Dashboard design choices

Misreading to prevent	Design choice that prevents it
“The tallest bar is the segment to target” (for example Age 30–35 at 36.1%)	Whiskers that span the reference line; segment size in every label (a long whisker on n = 216); a zero-based, fixed 0–50% axis; natural ordering instead of ranking
“The one interval clear of zero is a real problem”	Bonferroni-adjusted intervals and a subtitle stating the number of comparisons
“The model is worse than random” (0.496 < 0.5)	A shuffled-center reference line and the real result drawn inside the cloud
“The model narrowly missed a pass mark”	The bar is labeled “pre-set”, and the highest shuffled run (0.546) came within 0.004 of it

The KPI strip at the top of the dashboard shows 2,000 customers, a 31.05% overall churn rate, 0 of 31 segments differing (adjusted), and a cross-validated AUC of 0.496. Each is calculated from the exported data, not typed in. A caption states that the dashboard describes churn rates and does not measure return on retention spending.

9. Impact

The finding changes what the business should do next, even though it is a null result:

- **It prevents a wasted build.** A churn-targeting model on these seven features would score at chance (AUC 0.496) and its top-10% list would catch fewer churners than random selection (lift 0.93). Building and deploying it would cost analyst time and campaign budget for no better targeting.
- **It protects against acting on a spurious segment.** Without the correction and the intervals, the 30–35 age band (36.1% against 31.05%) looks like an actionable high-risk group. The analysis shows it is the expected tallest bar among 31, and its gap is within noise.
- **It reframes the question** from “who do we target?” to “does an offer work at all, and what data would let us target it?”, which the data cannot answer and a pilot can.

Setting the size of a pilot. At a 31% baseline with 80% power and a two-sided 5% test, detecting a 5-point reduction in churn takes about 1,300 customers per group (about 1,400 for a 5-point increase), a 3-point reduction about 3,600 per group, and a 2-point reduction about 8,200 per group. The whole dataset holds 2,000 customers, so a pilot of similar size could reliably detect only a large effect.

The break-even rule for any offer: it pays only if the reduction in churn probability it causes, multiplied by the value of a retained customer, exceeds the cost of the offer per customer. This dataset contains none of those three numbers: no customer value, no offer cost and no measured uplift.

10. Limitations and Ethics

Statistical limits

- **No detectable effect is not proof of no effect.** The 99.84% segment intervals are roughly ±6.5 to ±7.6 points wide for 500-customer segments and wider for smaller ones, so effects of a few points could be present and invisible.
- **Adjusted intervals are conservative.** The Bonferroni correction makes real small differences harder to see; the conclusion is “not detected at a conservative threshold”, not “does not exist”.

- **Small test set.** The held-out test set has 400 customers and about 124 churners, so its AUC has a standard deviation near 0.031 under no signal. It confirms the cross-validation but is one sample.
- **Thirty shuffles are few.** They cannot say how rarely noise reaches 0.55 more precisely than "about once in a few hundred runs".
- **The two dashboard views use different intervals** (adjusted 99.84% in the gap view, unadjusted 95% in the rate view); the subtitles say so.

Data limits

- **Provenance is unknown**, and the near-zero relationships between every feature and churn are consistent with randomly generated data. Conclusions describe this file, and may not transfer to a real customer base.
- **One snapshot.** There is no time dimension, so timing of purchases, offers and churn cannot be examined, and nothing here can establish cause and effect.
- **Missing information.** The dataset has no customer value, offer cost, recency of last purchase or support history, so profitability cannot be assessed and the null result may reflect missing predictors.

Ethical considerations

- **Fairness.** Age and gender are among the available features. Using them to decide who receives retention offers could treat customer groups unequally, and may raise legal or reputational concerns depending on jurisdiction. The finding that they carry no detectable signal argues against using them for this purpose.
- **Acting on noise.** Singling out a segment on the strength of a chance difference (for example the 30–35 age band) would mean treating real customers differently for no evidence-based reason.
- **Transparent communication.** The dashboard states the number of comparisons and describes intervals as conservative, so a viewer cannot easily read a null result as "the data shows no effect" or one spike as a finding.
- **Privacy.** Any extension that adds behavioral fields such as purchase recency or support-ticket counts should be reviewed for data protection and consent before use.

11. Recommendation

> **Decision:** Should the retention team use these customer attributes to choose who receives a retention offer?

Recommendation. Do not build or deploy a churn-targeting model using the current seven features. Run a randomized retention-offer pilot, and add behavioral fields that this dataset lacks, such as last-purchase date and support-ticket count, before reassessing modeling. Those fields are a hypothesis to test, not a finding: nothing here shows they would predict churn, and it is possible that churn in this data is simply unpredictable.

Evidence. None of the 31 tested segments showed a statistically detectable difference in churn after correcting for multiple comparisons. A model using all seven features had a cross-validated AUC of 0.496, not distinguishable from models trained on shuffled labels ($p = 0.645$). Its top-10% lift was 0.93, so the customers ranked highest risk did not churn at a higher rate than average.

What this analysis cannot answer. Small churn differences of a few percentage points may still exist. The dataset lacks customer value, offer cost and timing, so it cannot say whether a retention offer would be profitable.

Next step. Randomly assign eligible customers to receive the offer or not, then compare churn over the same follow-up period. Detecting a 5-point difference at 80% power needs roughly 1,300 to 1,400 customers per group, and smaller effects need many more. Owner and timeline: to be assigned by the retention lead. Still to be decided: the eligibility rule, the follow-up window (for example 30, 60 or 90 days), and the early-stopping rule.

What would change this recommendation. A new dataset in which features show a clear, repeatable association with churn, or a pilot showing a churn reduction large enough to justify the cost of the offer per customer.

Alternatives considered

Option	Evidence for it	Main risk	Verdict
A. Build a model on these features	None (AUC 0.496)	Spending to get no better than random	Not supported
B. Offer everyone an incentive	Depends on offer effect, not in this data	Paying customers who would have stayed	Reasonable only if offers are cheap
C. Randomized pilot	Would measure the offer's effect	Needs time and a large enough sample	**Recommended**
D. Do nothing	N/A	Churn continues at about 31%	Reasonable if offers are costly
E. Collect better data first	Current features carry no detectable signal	Delay; new fields may not help	**Recommended**, in parallel

What to improve next time, and what others can reuse

1. **Name the decision first.** Write the choice and its cost in one sentence before touching data.
2. **Fix the bars before the results.** Significance thresholds, model bars and segment definitions were set in advance, which is what makes a null result credible.
3. **Reconcile everything.** Counts that sum to the total, weighted gaps that sum to zero, and SQL and Python that agree catch silent bugs (lost customers, shifted bands, mislabeled buckets, a window function counting grouped rows).
4. **Compare with chance, not with zero.** Shuffle the labels and ask what noise scores before calling a model good; use AUC and lift against a stated baseline, not accuracy.
5. **Report what the data can rule out.** Intervals in standardized units show that large effects are excluded and small ones are not, which is more useful than "not significant".
6. **Guard against stale state.** In notebooks, restart and run all cells before exporting, and use assertions to stop a run when a key value changes.
7. **Next time:** use a larger sample or a time-stamped dataset, add behavioral features, run more permutation shuffles (about 200), and examine the dashboard with a real stakeholder.

Appendix A. Segment Export for Tableau (Views 1 and 2)

Builds one row per segment (31 rows) with counts, churn rates, Wilson intervals, gaps against all other customers, and unadjusted and Bonferroni-adjusted Newcombe intervals. Requires `statsmodels`.

```
from statsmodels.stats.proportion import proportion_confint, confint_proportions_2indep

N_SEGMENTS  = 31
ALPHA_UNADJ = 0.05
ALPHA_ADJ  = 0.05 / N_SEGMENTS  # Bonferroni, about 0.0016 (99.84% intervals)
OVERALL_N       = len(df)
OVERALL_CHURNED = int(df["Churn"].sum())
OVERALL_RATE    = OVERALL_CHURNED / OVERALL_N

# Segment definitions, fixed from the features alone (never from churn rates)
segments = {}
segments["Age"] = pd.cut(df["Age"], bins=[18,24,30,36,42,48,54,60,66,70], right=False,
                         labels=["18-23","24-29","30-35","36-41","42-47",
                                 "48-53","54-59","60-65","66-69"])
segments["Membership_Years"] = pd.cut(df["Membership_Years"], bins=[0,2.5,5.5,8.5,10.0001],
                         right=False, labels=["[0,2.5)","[2.5,5.5)","[5.5,8.5)","[8.5,10]"])
for col in ["Annual_Income", "Spending_Score", "Online_Purchases", "Discount_Usage"]:
    segments[col] = pd.qcut(df[col], 4, labels=["Q1 (lowest)", "Q2", "Q3", "Q4 (highest)"])
segments["Gender"] = df["Gender"].astype("category")

rows = []
for feature, seg in segments.items():
    assert seg.isna().sum() == 0, f"unassigned customers in {feature}"
    for order, level in enumerate(seg.cat.categories, start=1):
        mask = (seg == level)
        n, churned = int(mask.sum()), int(df.loc[mask, "Churn"].sum())
        rest_n, rest_churned = OVERALL_N - n, OVERALL_CHURNED - churned      # everyone else
        w_low, w_high = proportion_confint(churned, n, alpha=ALPHA_UNADJ, method="wilson")
        g_low, g_high = confint_proportions_2indep(churned, n, rest_churned, rest_n,
                            method="newcomb", compare="diff", alpha=ALPHA_UNADJ)
        a_low, a_high = confint_proportions_2indep(churned, n, rest_churned, rest_n,
                            method="newcomb", compare="diff", alpha=ALPHA_ADJ)
        rows.append({"feature": feature, "segment": str(level), "segment_order": order,
            "n": n, "churned": churned, "rate": churned / n,
            "rest_n": rest_n, "rest_churned": rest_churned, "rest_rate": rest_churned / rest_n,
            "ci_low": w_low, "ci_high": w_high, "gap": churned / n - rest_churned / rest_n,
            "gap_low": g_low, "gap_high": g_high, "gap_low_adj": a_low, "gap_high_adj": a_high,
            "overall_rate": OVERALL_RATE})
export = pd.DataFrame(rows)

assert len(export) == N_SEGMENTS
for f, g in export.groupby("feature"):
    assert g["n"].sum() == OVERALL_N  # every feature's segments sum to 2,000
export["excl_zero_unadj"] = (export["gap_low"] > 0)     | (export["gap_high"] < 0)
export["excl_zero_adj"]   = (export["gap_low_adj"] > 0) | (export["gap_high_adj"] < 0)
export.to_csv("churn_segments_export.csv", index=False)  # feeds Tableau views 1 and 2
```

Appendix B. AUC Export for Tableau (View 3)

Run after the permutation cell, so that `shuffled_means`, `cv_scores` and `test_auc` hold the real-label results. Writes 32 rows: 30 shuffled runs and two real results labeled so they cannot be confused.

```
BAR = 0.55                                              # pre-set bar
real_cv_mean  = cv_scores.mean()
n_shuffles    = len(shuffled_means)
n_at_or_above = int((shuffled_means >= real_cv_mean).sum())
p_empirical   = (n_at_or_above + 1) / (n_shuffles + 1)

shuffled_rows = pd.DataFrame({"result_type": "Shuffled labels (noise)",
                              "run": np.arange(1, n_shuffles + 1), "auc": shuffled_means})
real_rows = pd.DataFrame({"result_type": ["Real model, cross-validated",
                                          "Real model, test set (scored once)"],
                          "run": [np.nan, np.nan], "auc": [real_cv_mean, test_auc]})
export_auc = pd.concat([shuffled_rows, real_rows], ignore_index=True)
```

```
export_auc["preset_bar"]  = BAR  # reference values repeated on each row
export_auc["shuffled_center"] = shuffled_means.mean()  # so Tableau can draw lines and labels
export_auc["n_shuffles"]      = n_shuffles
export_auc["n_reaching_bar"]  = int((shuffled_means >= BAR).sum())
export_auc["empirical_p"]     = p_empirical
assert len(export_auc) == n_shuffles + 2
export_auc.to_csv("auc_vs_shuffled_export.csv", index=False)      # feeds Tableau view 3
```